C#基础（2）

问题

1. 面向对象思想
2. 命名规范文档
3. ref和out
4. 基类和派生类转换
5. 继承方法调用优先级
6. PMAlign工具中所有属性的用法
7. 哪些设置会影响执行时间

1 类

类成员：类上的字段、属性、方法和事件统称为类成员

类定义对象类型，但不是对象本身

1.1 类的成员

成员	描述
字段	字段是在类范围声明的变量。 字段可以是内置数值类型或其他类的实例。 例如，日历类可能具有一个包含当前日期的字段。
常量	常量是在编译时设置其值并且不能更改其值的字段。
属性	属性是类中可以像类中的字段一样访问的方法。 属性可以为类字段提供保护，以避免字段在对象不知道的情况下被更改。
方法	方法定义类可以执行的操作。 方法可接受提供输入数据的参数，并可通过参数返回输出数据。 方法还可以不使用参数而直接返回值。
事件	事件向其他对象提供有关发生的事情（如单击按钮或成功完成某个方法）的通知。 事件是使用委托定义和触发的。
运算符	重载运算符被视为类型成员。 重载运算符时，将其定义为类型中的公共静态方法。 有关详细信息，请参阅运算符重载。
索引器	使用索引器可以用类似于数组的方式为对象建立索引。
构造函数	构造函数是首次创建对象时调用的方法。 它们通常用于初始化对象的数据。
终结器	C# 中很少使用终结器。 终结器是当对象即将从内存中移除时由运行时执行引擎调用的方法。 它们通常用来确保任何必须释放的资源都得到适当的处理。
嵌套类型	嵌套类型是在其他类型中声明的类型。 嵌套类型通常用于描述仅由包含它们的类型使用的对象。

1.1.1 属性

1.1.1.1 属性的声明及使用

声明及使用属性，其中Age为属性，get和set为访问器：

```
class Person
{
    private int age;
    public int Age
    {
        get
        {
            return age;
        }
        set
        {
            age = value; // value为用户指定的值
        }
    }
}
class Program
{
    static void Main(string[] args)
    {
        Person person = new Person();
        person.Age = 18;
        int age = person.Age;
        Console.WriteLine(age); // 输出18
    }
}
```

等价写法：

```
public int Age { get; set; } // 编译器生成支持该属性的字段的存储位置，编译器自动实现get和set访问器的正文
public int Age { get; set; } = 18 // 设置指定默认值18，C#6及更高版本支持
public int Age // 表达式主体定义
{
    get => age;
    set => age = value;
}
```

1.1.1.2 属性的自动实现

编译器自动实现的属性在编译后，实质是生成了老式的读写属性（*《C# 4.0权威指南》9.4.5属性*）：

1.1.1.3 属性的常见用法

1. 验证，确保属性在允许范围内

```
class Person
{
    private int age;
    public int Age
    {
        get => age;
        set => age = value >= 18 ? value : throw new ArgumentException("未成年"); // 不允许设置年龄小于18周岁
    }
}
```

2. 只读，限制对属性的更改

```
class Person
{
    private int age;
    public int Age { get; } = 18;
}
```

代码编译后在构造函数中为属性赋值

- 读写属性（get + set）
- 只读属性（只有get）
- 只写属性（只有set，限制对敏感数据的访问）

1.1.2 构造函数

1. 无参数构造函数：没有声明显式构造函数时，该构造函数根据初始值（没有设置初始值就初始化为默认值）设定项来初始化字段和属性
2. 静态构造函数：无参数构造函数，用于初始化静态成员。不能直接调用，由CLR自动调用
3. 主构造函数（C#12开始）：将参数放在类的声明当中，参数值可以传递给字段、属性、方法、构造函数

```
public class NamedItem(string name)
{
    public string Name => name;
}
```

4. 私有构造函数：使用private声明的构造函数。在有私有构造函数的情况下，类无法被实例化（调用静态方法创建对象除外）。

```
class Base
{
    public int X { get; set; }
    private Base() { }
    public static Base Method()
    {
        return new Base();
    }
}
class Program
{
    static void Main(string[] args)
```

```
    {
        Base b = Base.Method();
        Console.WriteLine(b.X);
    }
}
```

如果类中只提供静态方法和字段来实现功能，使用私有构造函数是个不错的选择，如System.Math有静态字段E、PI和如下静态方法，没有必要为这些字段和方法创建Math类的实例：

```
public static int Abs (int value);
public static double Acos (double d);
public static double Asin (double d);
public static double Atan (double d);
……
```

1.1.2.1 构造函数中的关键字

this：在一个构造函数中调用另一个构造函数

base：在派生类的构造函数中调用基类构造函数

```
class Base
{
    private int x, y;
    public Base()
        : this(0, 0)
    {
        Console.WriteLine("基类中的无参构造函数");
    }
    public Base(int x, int y)
    {
        Console.WriteLine("基类中的有参构造函数");
    }
}
class Derived : Base
{
    private int x, y;
    public Derived()
        : base()
    {
        Console.WriteLine("派生类中的构造函数");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Base b = new Base();
        Console.WriteLine();
        Derived d = new Derived();
    }
}
```

运行结果：

```
基类中的有参构造函数
基类中的无参构造函数

基类中的有参构造函数
基类中的无参构造函数
派生类中的构造函数
```

1.1.2.2 实例化类的执行顺序

```
class Base
{
    private static Print p1 = new Print("基类的静态字段");
    private Print p2 = new Print("基类的实例字段");
    static Base()
    {
        Console.WriteLine("基类的静态构造函数");
    }
    public Base()
    {
        Console.WriteLine("基类的构造函数");
    }
}
class Derived : Base
{
    private static Print p1 = new Print("派生类的静态字段");
    private Print p2 = new Print("派生类的实例字段");
    static Derived()
    {
        Console.WriteLine("派生类的静态构造函数");
    }
    public Derived()
    {
        Console.WriteLine("派生类的构造函数");
    }
}
class Print
{
    public Print(String str)
    {
        Console.WriteLine(str);
    }
}
class Program
{
    static void Main(string[] args)
    {
        Derived c = new Derived();
        Console.WriteLine();
        Derived cc = new Derived();
    }
}
```

运行结果：

```
派生类的静态字段
派生类的静态构造函数
派生类的实例字段
基类的静态字段
基类的静态构造函数
基类的实例字段
基类的构造函数
派生类的构造函数

派生类的实例字段
基类的实例字段
基类的构造函数
派生类的构造函数
```

- 有静态字段时应遵循“先公后私”，并且静态构造函数紧跟着用来初始化静态字段
- 派生类实例被构造之前需要知道基类信息，因此基类构造函数先执行
- 静态成员只在第一次实例化时执行，以后再实例化都不执行

1.1.2.3 代码膨胀问题

在类中声明字段的时候就初始化，实际上对应的IL代码在运行时会将其赋值操作搬到构造方法内部执行

想象一种情况，如果持续增加字段，并且在声明时就对每个字段进行初始化赋值，同时持续增加新的构造函数，这时每个构造函数会把所有字段的初始化代码搬到其内部执行，这将会导致最终编译后的IL代码非常多，也就是所谓的**代码膨胀**。

示例：

```cpp
class Base
{
    private int x = 3;
    private double y = 3.3;
    private string s = "33";
    public Base()
    { }
    public Base(int x)
    {
        this.x = x;
    }
    public Base(double y)
    {
        this.y = y;
    }
    public Base(string s)
    {
        this.s = s;
    }
}
```

示例对应IL代码：https://sharplab.io/#v2:C4LglgNgNAJiDUAfAAgJgIwFgBQaAEAQgIYDOApjgN45614AOATmAG5HBI5gB2weAHngC8eAMwBuGnSat2nGAHsArgCMInAJ7CxAOglTaMthzzJ0ABjwltAIlGibk7HVOjCpMgAoAlAbyU8AF8/ZDdick8ePn5fZzpqOJdaYAALMBIdQRF+JxdgxNd3CMVVdTwNWJcEpLpU9J0tEQ1cunyXUKKvM0sSSvi/FzqM6xESFtp8wKA==

解决方法：把字段初始化全部放到一个无参构造函数内进行，让其他的构造函数显式调用该无参构造函数。

优化后的示例：

```
class Base
{
    private int x;
    private double y;
    private string s;
    public Base()
    {
        x = 3;
        y = 3.3;
        s = "33";

    }
    public Base(int x)
        : this()
    {
        this.x = x;
    }
    public Base(double y)
        : this()
    {
        this.y = y;
    }
    public Base(string s)
        : this()
    {
        this.s = s;
    }
}
```

优化后的示例对应IL代码：[https://sharplab.io/#v2:C4LgIgNgNAJiDUAfAAgJgIwFgBQaAEAQgIYDOApjgN45614AOATmAG5HBl5gB2weAHgG4adJq3acYAewCuAIwicAnsOx0GzNhzzJ0ABjwlV65AGZCpMgAoAlCNrU16uvzwBePKePPaS954A6L3tnEn8AIlNTcNUQgF8QswtyKx4+fjsnHxA8YAALMBJbEMcfWnzCgNcPIXjE82IU6XlFPCVMsrwciqKOulKynoC/DxU6rJ0GyytdAxI+7NyC3pKQ9SGwjyNxuKA=]

1.2 类的继承

1.3 静态类

特点：

1. 只包含静态成员
2. 不包含实例构造函数
3. 会进行密封（无法派生）
4. 无法实例化。编译器进行检查，确保不会意外地添加任何实例成员，比私有构造函数更严格

1.5 访问修饰符

调用方的位置	`public`	`protected internal`	`protected`	`internal`	`private protected`	`private`
在类内	✔	✔	✔	✔	✔	✔
派生类（相同程序集）	✔	✔	✔	✔	✔	✘
非派生类（相同程序集）	✔	✔	✘	✔	✘	✘
派生类（不同程序集）	✔	✔	✔	✘	✘	✘
非派生类（不同程序集）	✔	✘	✘	✘	✘	✘

`protected`：同一class或派生的class中代码可访问

`internal`：同一程序集中的任何代码都可以访问该类型或成员，但其他程序集中的代码不可以。

1.5.1 默认访问级别

1. 直接在命名空间中声明的类、记录和结构（即，没有嵌套在其他类或结构中的类、记录和结构），默认public
2. 类成员和结构成员（包括嵌套的类和结构），默认private，构造函数默认public
3. 在命名空间内直接声明的接口，默认internal，其成员默认public
4. 枚举成员只能为public

2 结构体与类的区别

1. 值类型与引用类型的区别
    1. 值类型数据存栈上；引用类型中引用存栈上，数据存堆上
    2. 值类型变量赋值时直接复制包含的值，在栈中开辟新的空间保存该值，并且与原变量互不影响；引用类型变量赋值时只复制变量的引用，原变量值改变时会同步
    3. 值类型作为字段时跟随所属实例存储，作为局部变量时存储在栈上；引用类型不存在跟随问题
    4. 值类型内存管理效率高，不支持多态，适合做存储数据的载体；引用类型支持多态，适合用于定义应用程序的行为
    5. 值类型都是密封的，不能派生出新的类型；引用类型可以
    6. 值类型不能包含null值；引用类型可以
2. 结构体中声明的字段无法赋初值；类可以

```
struct MyStruct
{
    int x = 1; // 报错
}
```

3. 在 C# 10 之前，结构体不能包含显式无参数构造函数，因为编译器会自动提供一个，现在可以（并且声明有参构造函数后能把无参构造函数顶掉）；类一直可以包含显式无参构造函数

```
struct MyStruct
{
    public int x;
    public MyStruct()
    {
        x = 3;
    }
}
class Program
{
    static void Main(string[] args)
    {
        MyStruct ms = new MyStruct();
        Console.WriteLine(ms.x); // 输出3
    }
}
```

在结构体中，在变量声明时就初始化也会像类一样将初始化的IL代码搬到构造函数内部执行

4. 结构体中无法定义终结器（析构函数）；类可以

```
struct MyStruct
{
    ~MyStruct() { } // 报错，只有类类型才能包含析构函数
}
```

5. 结构体不能被继承；类可以
6. 无静态结构体；有静态类
7.